

第 49 讲：Sequence 与 RNN

Hidden State 到底怎样把过去的信息带到下一时刻

《面向 LLM / Agent Systems 科研的深度学习零基础课》

本讲定位

前 48 讲已经完成了深度学习最基础的数值计算、优化、泛化、Normalization 和 CNN。本讲开始从“空间结构”转向“序列结构”。我们的目标不是把 RNN 学成一个独立研究方向，而是把它学到足以回答三个问题：

1. Hidden State 到底是什么，为什么它能把历史带到下一时刻？
2. 为什么同一套参数要沿时间反复使用？
3. 为什么这种递推结构会带来 BPTT、梯度消失/爆炸和难以并行的问题？

理解这些问题以后，下一讲学习 LSTM / GRU 才不会停留在“多了几个门”，再下一步才能真正理解 Transformer 为什么改变了序列建模方式。

0. 学完以后，你应该能做到什么？

- 解释 Sequence、Time Step、Hidden State、Recurrent Weight 的含义。
- 手算一个最小 RNN 的 h_1, h_2, h_3 。
- 解释为什么同一个当前输入，在不同历史下会得到不同 Hidden State。
- 看懂 RNN 的参数 Shape，并解释为什么参数量不随 Sequence Length 增长。
- 看懂 PyTorch `nn.RNN` 的输入输出 Shape。
- 解释 Many-to-one、Many-to-many 等最基本序列任务。
- 理解 BPTT 是“把时间展开后的计算图做 Backpropagation”。
- 用连续乘积解释 Vanishing / Exploding Gradient。
- 解释为什么 RNN 的时间依赖会限制训练并行性。
- 为下一讲 LSTM / GRU 和后面的 Transformer 做好结构基础。

【科研连接】

你以前在 LLM 课程中已经见过 RNN / LSTM，也知道 Transformer 后来取代了它们成为语言模型主流。但“知道历史”不等于理解原因。本讲会把 RNN 拆到底层：**序列信息不是神秘地被“记住”，而是被压进 Hidden State，并通过同一个递推公式一时刻一时刻传下去。**

1 什么是 Sequence? 为什么顺序本身就是信息?

Sequence (序列) 就是一串有顺序的数据:

$$x_1, x_2, x_3, \dots, x_T$$

其中 T 是 Sequence Length, 每一个位置称为一个 **Time Step (时间步)**。

序列和普通表格数据最大的区别之一是: **交换顺序通常会改变含义**。

例如:

- 文本: “我不喜欢” 与 “我喜欢不” 不同;
- 股票价格: 今天、昨天、前天的顺序不能乱;
- Agent trajectory: Observation $\rightarrow$ Action $\rightarrow$ Tool Result $\rightarrow$ Next Action 的先后关系直接决定系统状态。

【通俗解释】

对一个普通 MLP 来说, 如果你把一条序列简单 Flatten 成一个大向量, 它并不会自动理解 “第 1 个位置先发生、第 2 个位置后发生”。RNN 的核心目标就是: **让当前计算显式依赖上一时刻的内部状态**。

2 最小 RNN: 当前输入 + 上一时刻 Hidden State

最基本的 RNN 可以写成:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$$

这里:

- x_t : 当前时间步输入;
- h_{t-1} : 上一时间步 Hidden State;
- W_x : 当前输入到 Hidden State 的 Weight;
- W_h : 上一 Hidden State 到当前 Hidden State 的 Recurrent Weight;
- b : Bias;
- h_t : 当前新的 Hidden State;
- $\tanh$: 非线性激活。

【原理】

RNN 的 “记忆” 并不是另外存了一份完整历史。它做的是:

$$\text{过去的信息} \rightarrow h_{t-1} \rightarrow \text{参与当前计算} \rightarrow h_t$$

因此 Hidden State 是一种**递推压缩状态**。

3 先从 1 维标量 RNN 手算：每个数字到底发生了什么？

为了避免一开始就被矩阵淹没，我们先用标量版本：

$$h_t = \tanh(w_x x_t + w_h h_{t-1} + b)$$

设：

$$w_x = 0.8, \quad w_h = 0.5, \quad b = -0.1, \quad h_0 = 0$$

输入序列：

$$x = [1, 2, -1]$$

第 1 步：

$$z_1 = 0.8 \times 1 + 0.5 \times 0 - 0.1 = 0.7$$

$$h_1 = \tanh(0.7) \approx 0.6044$$

第 2 步：

$$z_2 = 0.8 \times 2 + 0.5 \times 0.6044 - 0.1 \approx 1.8022$$

$$h_2 = \tanh(1.8022) \approx 0.9470$$

第 3 步：

$$z_3 = 0.8 \times (-1) + 0.5 \times 0.9470 - 0.1 \approx -0.4265$$

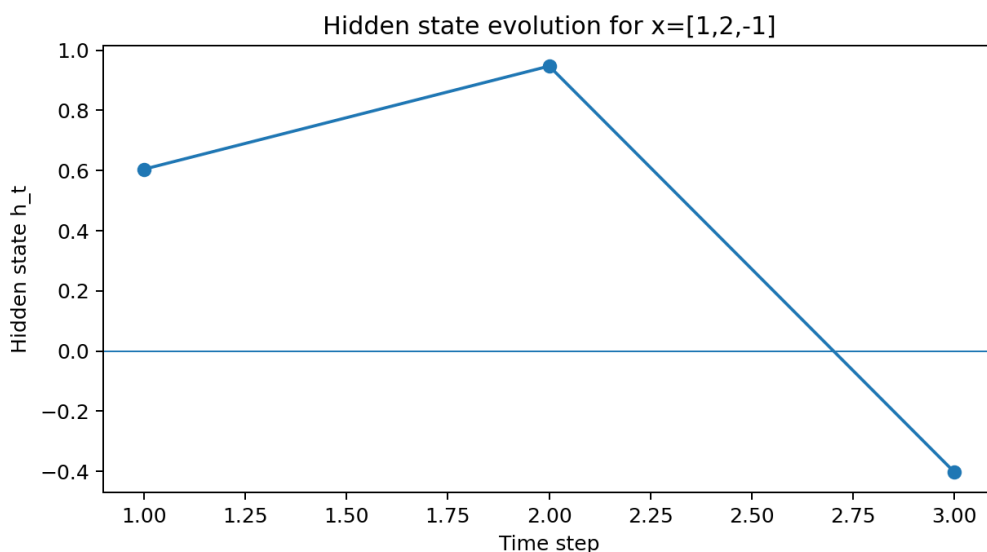
$$h_3 \approx -0.4024$$

【具体数字例子】

注意第 2 步的输入虽然只是 $x_2 = 2$ ，但 h_2 并不是只由 2 决定，因为 h_1 已经携带了第 1 步的信息。第 3 步同理。因此：

$$h_t = f(x_t, h_{t-1})$$

正是“序列历史进入当前表示”的最小机制。



4 Hidden State 真能表示历史吗？做一个关键对照

考虑两条序列：

$$A = [2, 2, 0.5]$$

$$B = [-2, -2, 0.5]$$

最后一个当前输入都是：

$$x_3 = 0.5$$

但因为前两步不同，实验得到：

$$h_3^{(A)} \approx 0.6529$$

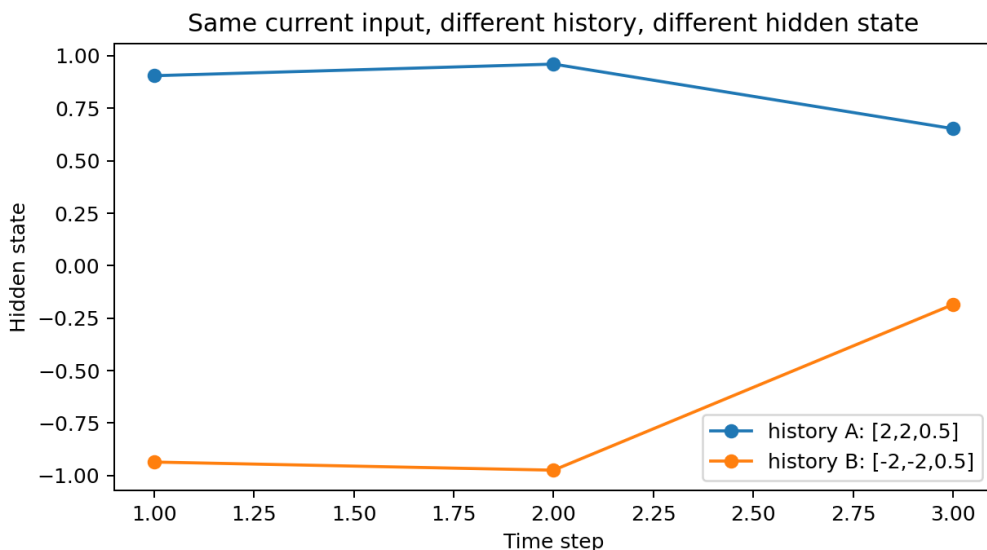
$$h_3^{(B)} \approx -0.1849$$

差值约：

$$0.8378$$

【通俗解释】

如果模型只看“当前输入 0.5”，两条序列这一刻应该完全相同。但 RNN 仍输出不同 Hidden State，因为上一时刻的状态不同。这就是“历史进入当前表示”的直接证据。



5 为什么叫 Recurrent? 同一个 Cell 在时间上反复使用

把 RNN 沿时间展开：

$$h_1 = f(x_1, h_0; \theta)$$

$$h_2 = f(x_2, h_1; \theta)$$

$$h_3 = f(x_3, h_2; \theta)$$

$\vdots$

$$h_T = f(x_T, h_{T-1}; \theta)$$

关键在于所有时间步使用的是**同一套参数** θ 。

对于最简单单层 RNN：

$$W_{ih} \in \mathbb{R}^{H \times D}$$

$$W_{hh} \in \mathbb{R}^{H \times H}$$

以及两组 Bias (PyTorch 实现中)：

$$b_{ih}, b_{hh} \in \mathbb{R}^H$$

参数量：

$$HD + H^2 + 2H$$

它与 Sequence Length T 无关。

实验中 $D = 4, H = 6$ ：

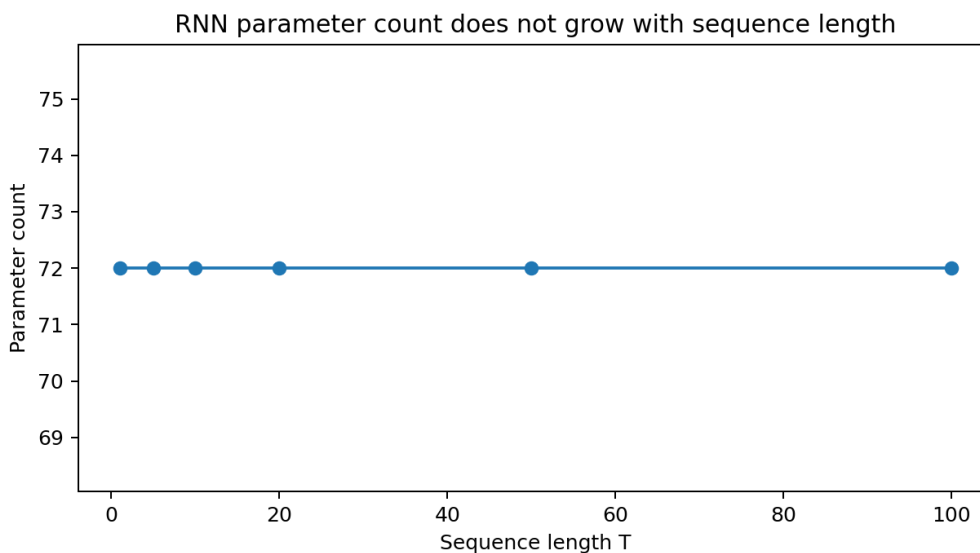
$$6 \times 4 + 6 \times 6 + 2 \times 6 = 72$$

无论：

$$T = 5, 20, 100$$

参数量始终都是：

$$72$$



【科研连接】

这和 CNN 的参数共享非常相似，但共享维度不同：CNN 在**空间位置**共享 Kernel，RNN 在**时间位置**共享同一套 Recurrent Cell 参数。

6 矩阵版 RNN：从一个标量升级到 Hidden Vector

现实 RNN 的 Hidden State 通常不是一个数，而是长度为 H 的向量：

$$h_t \in \mathbb{R}^H$$

输入：

$$x_t \in \mathbb{R}^D$$

于是：

$$h_t = \tanh(W_{ih}x_t + W_{hh}h_{t-1} + b)$$

Shape 对齐：

$$(H, D) @ (D,) \rightarrow (H,)$$

$$(H, H) @ (H,) \rightarrow (H,)$$

然后两者与 Bias 相加，仍得到：

$$(H,)$$

最后经过 $\tanh$ ，Shape 不变。

【原理】

所以 RNN 并没有引入全新的数学。它只是把你已经学过的：

Linear + Activation

加上了一条来自上一个时间步的 Recurrent Connection。

7 PyTorch nn.RNN 的输入输出 Shape

如果使用：

`batch_first=True`

典型输入：

$X.shape = (B, T, D)$

输出：

$Y.shape = (B, T, H)$

最终 Hidden：

$h_n.shape = (L, B, H)$

其中 L 是层数（忽略双向时）。

实验：

$B = 3, T = 7, D = 5, H = 4, L = 2$

得到：

$X : (3, 7, 5)$

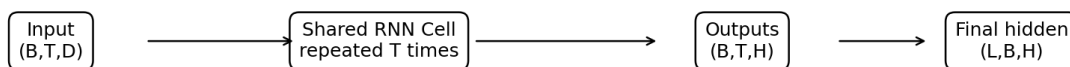
$Y : (3, 7, 4)$

$h_n : (2, 3, 4)$

【通俗解释】

Y 保留了所有时间步的 top-layer Hidden State； h_n 只保留每一层最后时间步的 Hidden State。

RNN tensor shape flow



【常见误区 / 风险】

如果 `batch_first=False`，PyTorch 默认更传统的 Shape 是 (T, B, D) 。以后遇到 RNN Shape 错误，第一件事要检查的就是 `batch_first`。

8 PyTorch 内部到底是不是同一个递推？手工与框架对齐

我们固定：

$W_{ih} = 0.8, \quad W_{hh} = 0.5$

并让两组 Bias 相加后等于：

$$-0.1$$

输入仍为：

$$[1, 2, -1]$$

手工递推输出：

$$[0.604368, 0.947032, -0.402379]$$

PyTorch `nn.RNN` 输出完全相同，最大绝对误差：

$$2.98 \times 10^{-8}$$

【代码实验】

```
rnn = nn.RNN(1, 1, batch_first=True)
with torch.no_grad():
    rnn.weight_ih_l0.fill_(0.8)
    rnn.weight_hh_l0.fill_(0.5)
    rnn.bias_ih_l0.fill_(-0.1)
    rnn.bias_hh_l0.zero_()

out, h_n = rnn(x, h0)
```

【通俗解释】

所以 `nn.RNN` 不是一个不可理解的“序列黑盒”。它只是替你把同一递推公式沿 T 个时间步自动执行，并帮你维护 Tensor Shape 与 Autograd 计算图。

9 RNN 可以怎样使用输出? Many-to-one 与 Many-to-many

RNN 并不规定任务必须只有一种输出方式。

1. Many-to-one

整条序列最后只输出一个结果，例如：

- 情感分类；
- 整段行为序列分类；
- Agent trajectory 是否成功。

常见做法是取最后 Hidden：

$$h_T \rightarrow \text{Linear} \rightarrow \text{logits}$$

实验中：

$$h_n[-1].\text{shape} = (B, H)$$

经过 Linear 后：

$$(B, H) \rightarrow (B, C)$$

2. Many-to-many

每个时间步都产生输出，例如：

- 序列标注；
- 每个 Token 的标签；
- 每个时刻预测下一个值。

这时通常使用完整：

$$Y.\text{shape} = (B, T, H)$$

再在时间维上分别投影到输出空间。

【科研连接】

早期 Language Model 就可以用 RNN 做 Next-token Prediction：每个时间步 Hidden State 预测下一个 Token。但 Hidden State 必须把前文信息压缩进一个固定维度状态，而且计算必须按时间顺序递推，这两个结构特征后来都成为 Transformer 改进的关键入口。

10 RNN 的“记忆”并不等于永久存储完整历史

这是初学者最容易产生的误解之一。

Hidden State 的维度固定：

$$h_t \in \mathbb{R}^H$$

无论前面有 5 个 Token，还是 500 个 Token，历史都必须不断压缩进入这 H 个数字。

所以 RNN 不是：

“把全部历史原样保存在 Hidden State 里。”

而更接近：

“每一步根据当前输入和过去的压缩摘要，重新计算一个新的压缩摘要。”

【常见误区 / 风险】

这也是为什么“Hidden State 有历史信息”与“Hidden State 能完美保存长期历史”是两个完全不同的结论。普通 RNN 尤其容易在长序列上丢失早期信息。

11 Training 时为什么要把 RNN 沿时间展开？

Forward 时看到的是递推：

$$h_t = f(h_{t-1}, x_t)$$

但为了做 Backpropagation，可以把它沿时间展开成一张深计算图：

$$h_0 \rightarrow h_1 \rightarrow h_2 \rightarrow \cdots \rightarrow h_T$$

同一组 W_{ih}, W_{hh} 在不同时间节点重复出现。

如果最终 Loss 依赖 h_T ，那么更早的参数贡献也会通过：

$$L \rightarrow h_T \rightarrow h_{T-1} \rightarrow \cdots \rightarrow h_1$$

往回传。

这就是：

$$\text{BPTT} = \text{Backpropagation Through Time}$$

【通俗解释】

BPTT 并不是一种完全不同的数学。它本质上仍是第 22-25 讲学过的 Chain Rule + Computational Graph，只不过“网络深度”现在有一部分来自时间轴。

12 为什么 RNN 容易梯度消失 / 爆炸？

先看最小线性递推：

$$h_t = ah_{t-1}$$

于是：

$$h_T = a^T h_0$$

并且：

$$\frac{\partial h_T}{\partial h_0} = a^T$$

如果：

$$a = 0.5, T = 30$$

则：

$$0.5^{30} \approx 9.31 \times 10^{-10}$$

几乎消失。

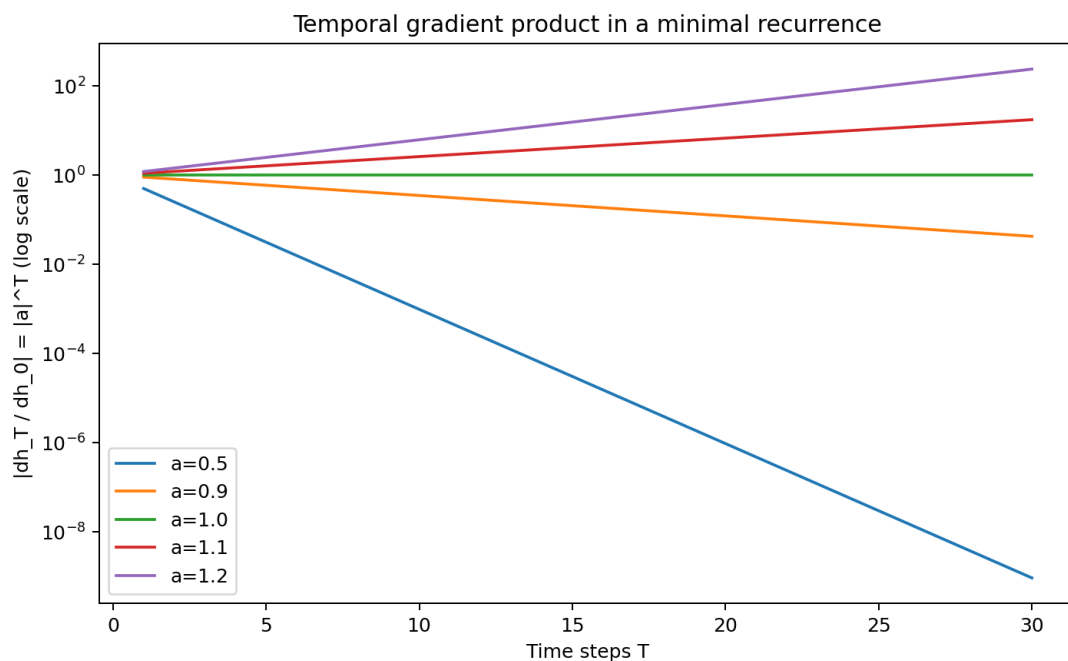
如果：

$$a = 1.2, T = 30$$

则：

$$1.2^{30} \approx 237.38$$

明显放大。



真实 RNN 中还有：

$$\tanh'(z) = 1 - \tanh^2(z)$$

这类局部导数参与连续相乘。

实验在 $a = 1$ 、20 步 $\tanh$ 递推中：

- $h_0 = 0.1$ 时， $dh_{20}/dh_0 \approx 0.829$ ；
- $h_0 = 1$ 时， ≈ 0.0161 ；
- $h_0 = 3$ 时，仅 $\approx 1.73 \times 10^{-4}$ 。

【原理】

早期时间步要影响最终 Loss, Gradient 必须跨过很多 Recurrent Jacobian / Activation Derivative。连续乘积系统性小于 1, 就会 Vanish; 系统性大于 1, 就可能 Explode。

13 Gradient Clipping 能解决 RNN 的长期记忆问题吗?

Gradient Clipping 对 Exploding Gradient 很有帮助, 例如限制:

$$\|g\| \leq c$$

可以避免某一步更新因为 Gradient 过大而直接炸掉。

但:

【常见误区 / 风险】

Gradient Clipping 不能把已经消失到接近 0 的长期 Gradient “重新变回来”。所以它主要缓解 Exploding Gradient, 不是长期依赖问题的完整解决方案。

这也是下一讲 LSTM / GRU 要引入更特殊状态通道与门控结构的重要原因。

14 为什么 RNN 难以沿时间并行?

观察:

$$h_t = f(x_t, h_{t-1})$$

要计算 h_t , 必须先知道:

$$h_{t-1}$$

而 h_{t-1} 又依赖 h_{t-2} 。

因此:

$$h_1 \rightarrow h_2 \rightarrow h_3 \rightarrow \cdots \rightarrow h_T$$

在时间维上存在硬依赖链。

【通俗解释】

即使 GPU 可以同时做很多矩阵乘法, RNN 的时间方向仍然像排队: 第 t 步必须等第 $t-1$ 步完成。Batch 维可以并行, 但 Time 维天然更难全部同时展开计算。

【科研连接】

这正是 Transformer 的一个巨大工程优势来源: Self-Attention 在训练阶段可以让同一层里的多个 Token 位置并行参与矩阵运算, 而不需要先完成 h_1 才开始 h_2 。当然, 自回归生成阶段仍然存在 Token-by-Token 的因果依赖, 这和“训练时层内并行”是两个不同问题。

15 RNN 与 Transformer：不要把结论简化成“旧模型 vs 新模型”

维度	RNN	Transformer / Self-Attention
序列信息	Hidden State 递推传递	Token 表示通过 Attention 直接交互
时间依赖	h_t 依赖 h_{t-1}	同层训练时各 Token 可并行
长期路径	早期信息需跨很多递推步骤	Attention 可形成更短的交互路径
参数共享	同一 RNN Cell 跨时间共享	同一层 Projection Weight 跨 Token 共享
状态瓶颈	历史持续压入固定 Hidden State	显式保留序列位置表示参与 Attention
训练并行	时间轴并行受限	更适合大规模矩阵并行

但不能据此说 RNN “完全没用”。在流式处理、严格在线状态更新、小模型、低延迟或特定时序任务中，RNN 类结构仍可能有价值。

16 与 LLM / Agent Systems 的直接关系

1. LLM 的历史背景

在 Transformer 之前，RNN/LSTM 是序列建模与 Language Modeling 的核心路线之一。理解 RNN 的限制，才能理解 Transformer 的设计动机不是凭空出现的。

2. Agent trajectory 本质上也是 Sequence

一个 Agent 运行轨迹可以写成：

$$(o_1, a_1, r_1, o_2, a_2, r_2, \dots)$$

即使今天我们更多使用 Transformer，状态如何随时间积累、历史如何影响当前决策仍然是 Agent Systems 的基本问题。

3. Stateful Agent 与 Hidden State 类比

Agent 的 memory / scratchpad / environment state 不是 RNN Hidden State，但二者共享一个系统科学问题：

“有限状态表示怎样保留过去中对未来真正有用的信息？”

这对你以后研究长期经历、persistent agents、functional heterogeneity 都有直接概念价值。

17 常见误区

【常见误区 / 风险】

误区 1：Hidden State 就等于完整历史。
不是。它是固定维度的递推压缩状态。

【常见误区 / 风险】

误区 2: Sequence Length 越长, RNN Parameter 越多。

不是。同一套 Weight 在时间上共享; T 增大主要增加计算量和计算图深度。

【常见误区 / 风险】

误区 3: RNN 不能并行, 所以 GPU 完全没用。

错误。Batch 维、矩阵乘法内部仍可以并行; 受限的是跨时间的递推依赖。

【常见误区 / 风险】

误区 4: Gradient Clipping 能解决 Vanishing Gradient。

不能。Clipping 主要控制过大的 Gradient, 不会自动恢复已经消失的长期信号。

【常见误区 / 风险】

误区 5: Transformer 取代 RNN 后, RNN 原理已经没有学习价值。

对你的目标而言仍有价值, 因为 RNN 是理解长期依赖、BPTT、状态压缩和 Transformer 设计动机的最好对照组之一。

18 科研阅读：以后看到 RNN 实验应该先问什么？

1. 输入 Shape、Hidden Size、层数是多少？
2. 单向还是 Bidirectional？
3. 取最后 Hidden, 还是使用每个 Time Step 的 Output？
4. Sequence Length 多长, 是否截断？
5. 是否使用 Truncated BPTT？
6. 是否做 Gradient Clipping? 阈值多少？
7. Initial Hidden State 如何处理？
8. Padding / Variable Length Sequence 怎样处理？
9. 比较 RNN / LSTM / Transformer 时, 参数量、训练预算和数据是否公平？
10. 评价“长期依赖”时, 任务是否真的需要跨很远时间的信息？

19 本讲总结：把 RNN 压成一条主线

第 49 讲核心链

Sequence $\rightarrow x_t \rightarrow (h_{t-1}, x_t) \rightarrow h_t \rightarrow$ Shared RNN Cell $\rightarrow$ BPTT $\rightarrow$ Temporal Gradient Product $\rightarrow$ Lo

你现在应该能够解释: RNN 的“记忆”为什么来自 Hidden State 递推; 为什么参数量不随 T 增长; 为什么 BPTT 会把时间轴变成深计算图; 为什么长序列中容易出现 Vanishing / Exploding Gradient; 以及为什么这种时间依赖会限制训练并行。

【科研连接】

下一讲：第 50 讲《LSTM 与 GRU：门控结构到底怎样缓解普通 RNN 的长期依赖问题》。重点不是背四个 Gate 的名字，而是理解 Cell State / Gate 如何改变“信息与 Gradient 必须反复经过同一个压缩变换”的结构。

5 分钟口述自检

不看讲义，尝试回答：

1. 写出最基本的 RNN Hidden State 更新公式，并解释每个变量。
2. 为什么同一个 x_t 在不同历史下可以产生不同 h_t ？
3. 为什么 Sequence Length 变长时参数量通常不增加？
4. `nn.RNN(batch_first=True)` 的输入和输出 Shape 是什么？
5. BPTT 与普通 Backpropagation 的关系是什么？
6. 为什么 a^T 能帮助理解梯度消失和爆炸？
7. 为什么 RNN 时间轴难以完全并行？

如果这 7 个问题能独立说清，本讲核心已经掌握。